

DNS Fundamentals & AWS Route 53 — Study Notes

Table of Contents

1. [What is DNS?](#)
2. [DNS Resolution Journey](#)
3. [Core DNS Terminology](#)
4. [DNS Infrastructure Components](#)
5. [Domain Registrars & DNS Providers](#)
6. [AWS Route 53 Overview](#)
7. [Route 53 Building Blocks](#)
8. [DNS Record Types Explained](#)
9. [Traffic Routing Strategies](#)
10. [Health Monitoring in Route 53](#)
11. [Hands-On Labs](#)
12. [Summary & Cheat Sheet](#)

What is DNS?

The **Domain Name System (DNS)** is essentially a massive, globally distributed directory that converts website names into numerical IP addresses that computers understand.

User types → `www.amazon.com`

DNS returns → `205.251.242.103`

Browser connects → Web page loads

Think of it this way — your smartphone contacts list stores names instead of making you memorize phone numbers. DNS does the exact same thing for the internet. Instead of remembering `172.217.18.36`, you just type `google.com`.

Why Does DNS Matter?

Problem Without DNS	Solution With DNS
Must memorize IPs like <code>54.239.28.85</code>	Just type <code>amazon.com</code>

Problem Without DNS	Solution With DNS
Changing server IPs breaks access	DNS record updates are transparent
No scalability for millions of sites	Distributed hierarchy handles global scale

DNS Resolution Journey

Every time you visit a website, a behind-the-scenes lookup process happens in milliseconds. Here's a breakdown of that process:

Step 1: You type "www.netflix.com" in browser
↓
Step 2: Browser Cache – Already visited before?
↓ (No)
Step 3: OS Cache – Stored in system?
↓ (No)
Step 4: Recursive Resolver (ISP or 8.8.8.8)
↓
Step 5: Root Server – "Where is .com?"
↓
Step 6: TLD Server (.com) – "Who manages netflix.com?"
↓
Step 7: Authoritative Server – "netflix.com = 54.74.12.1"
↓
Step 8: Browser connects → Page loads ✅
Step 9: Result cached based on TTL

Who Does What?

DNS Component	Real-World Analogy	Function
Recursive Resolver	A librarian who does the research for you	Performs full lookup on behalf of client
Root Name Server	Library's main index	Points to correct TLD server
TLD Server	A section divider in the library	Manages .com , .org , .net etc.
Authoritative Server	The actual book you needed	Holds and returns final DNS records
DNS Cache	Your personal bookmarks	Stores recent lookups for speed



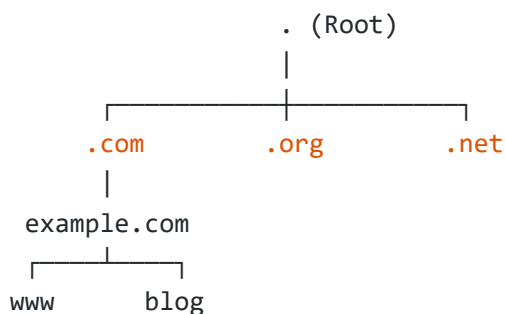
Core DNS Terminology

Term	Plain-English Meaning	Example
Domain Name	The website address users type	shopify.com
IP Address	Numeric address of the actual server	23.227.38.65
TTL (Time To Live)	How long a DNS record stays cached	300 seconds
FQDN	The full, complete domain name including the dot	www.shopify.com.
TLD	The ending part of a domain	.com , .in , .org
Zone File	A text file containing all DNS records	Records for example.com



DNS Infrastructure Components

How the Hierarchy Looks



The DNS system is like a tree — queries start at the root and travel down branches until they find the answer.



Domain Registrars & DNS Providers

The Difference Explained

Domain Registrar = Who owns the domain name

DNS Hosting Provider = Who answers DNS queries **for** that domain

These can be the same company or different companies — your choice.

Real Example:

- Buy domain `mystore.com` on **Namecheap**
- Host DNS records on **AWS Route 53**
- Point Namecheap nameservers → Route 53

Popular Options

Service	Category	Strength
GoDaddy	Registrar + DNS	Widely used, beginner-friendly
Namecheap	Registrar + DNS	Budget-friendly, includes privacy protection
Cloudflare	DNS + CDN	Extremely fast, DDoS protection included
AWS Route 53	Registrar + DNS	Deep AWS integration, advanced features

AWS Route 53 Overview

What Exactly is Route 53?

Route 53 is Amazon's **fully managed, highly available DNS and traffic routing service**. It's authoritative, meaning you have full control to create, edit, and delete DNS records directly.

Route 53 handles three things:

1. Domain Registration
2. DNS Record Management
3. Traffic Routing + Health Checks

Where Does the Name "Route 53" Come From?

"Route" → It routes internet traffic to destinations

"53" → DNS communicates over Port 53

Protocol	Port Number
HTTP	80
HTTPS	443

Protocol	Port Number
FTP	21
DNS	53 ←

What Makes Route 53 Special?

Capability	Description
100% Uptime SLA	AWS guarantees full availability
Global Anycast Network	DNS servers spread worldwide
Intelligent Routing	8 different routing strategies
Health Monitoring	Automatically detects and avoids broken servers
AWS Native	Works seamlessly with EC2, ALB, CloudFront, S3

🧱 Route 53 Building Blocks

📁 Hosted Zones

A Hosted Zone is like a folder that stores all DNS records for one domain. When you create a hosted zone for `example.com`, everything DNS-related for that domain lives inside it.

Hosted Zone: `example.com`

- └─ A Record → `www.example.com` → `54.1.2.3`
- └─ MX Record → `mail.example.com` → Google Mail
- └─ TXT Record → Verification codes
- └─ CNAME Record → `shop.example.com` → `shopify.com`

Two Types of Hosted Zones

Type	Accessible From	Use Case
🌐 Public Hosted Zone	Anywhere on the internet	Public-facing websites
🔒 Private Hosted Zone	Only inside a specific VPC	Internal apps, databases

When you create a hosted zone, Route 53 automatically generates an **NS record** and an **SOA record** for you.



Name Servers (NS)

Name Servers are the servers that actually hold and serve your DNS records. Route 53 assigns **4 name servers** to every hosted zone for redundancy.

Sample Route 53 Name Servers:

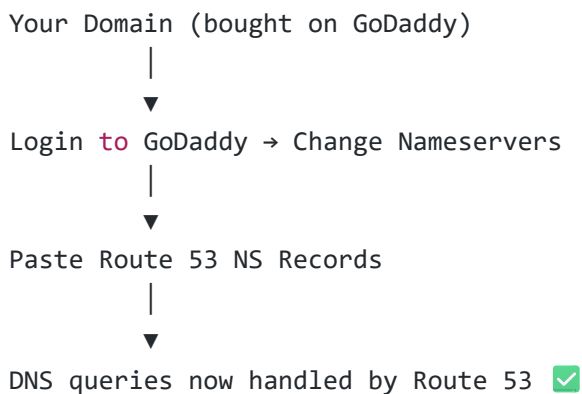
ns-245.awsdns-30.com
ns-1532.awsdns-07.co.uk
ns-831.awsdns-40.net
ns-1267.awsdns-30.org

Having 4 different name servers means even if one fails, the others keep answering queries.



DNS Delegation

Delegation is the process of telling the world "Route 53 is in charge of DNS for my domain."



Without delegation, Route 53 has the records but nobody knows to ask Route 53 for them.



SOA Record (Start of Authority)

The SOA record is the administrative ID card of your DNS zone. It tells other DNS servers basic facts about your zone.

Contains:

- Which server is the primary name server
- Admin contact email
- A serial number (zone version tracker)
- Timing settings for zone refreshes

Route 53 manages SOA records automatically — you rarely need to touch this.

TTL (Time To Live)

TTL is a number (in seconds) that tells DNS resolvers how long to remember a record before checking again.

TTL = 300 → Cache for 5 minutes → Faster to change, more queries

TTL = 86400 → Cache for 24 hours → Fewer queries, slow to update

Scenario	Recommended TTL
Making changes soon	Low (60–300 sec)
Stable, no changes planned	High (3600–86400 sec)
Before a migration	Drop to low TTL first

DNS Record Types Explained

Visual Overview

DNS Record Toolkit

	A	→ domain name	→ IPv4 address
	AAAA	→ domain name	→ IPv6 address
	CNAME	→ domain name	→ another domain name
	MX	→ domain name	→ mail server
	TXT	→ domain name	→ plain text data
	NS	→ domain name	→ name server
	PTR	→ IP address	→ domain name
	SRV	→ service name	→ host + port
	Alias	→ domain name	→ AWS resource

A Record

The most basic and commonly used DNS record. Maps a domain name to an IPv4 address.

Record Type : A

Name : mywebsite.com

Value : 13.234.56.78

TTL : 300

Simple example: Typing `mywebsite.com` → A record → browser connects to `13.234.56.78`

🟦 AAAA Record

Exactly like an A record but for IPv6 addresses (the newer, longer IP format).

```
Record Type : AAAA
Name       : mywebsite.com
Value      : 2400:cb00:2048:1::c629:d7a2
TTL        : 300
```

🟡 CNAME Record

Creates an alias from one domain name to another domain name.

```
blog.mywebsite.com → mywebsite.com
shop.mywebsite.com → stores.shopify.com
```

⚠️ Important Limitation:

❌ CANNOT use CNAME **for** the root domain
`mywebsite.com` → NOT allowed as **a** CNAME

✅ CAN use CNAME **for** subdomains
`www.mywebsite.com` → Allowed ✓

📧 MX Record

Tells email servers where to deliver emails for your domain.

```
Someone sends email to → hello@mywebsite.com
MX Record says          → Deliver to mail.google.com
Priority 1               → Try this server first
Priority 10              → Fallback server
```

Priority	Server	Role
1	aspmx.l.google.com	Primary mail server
5	alt1.aspmx.l.google.com	Secondary

Priority	Server	Role
10	alt2.aspmx.l.google.com	Tertiary

TXT Record

Stores plain text data attached to your domain. Primarily used for verification and email security.

Use Case	What It Does
Domain Verification	Proves you own the domain to Google/Microsoft
SPF	Lists which servers can send email on your behalf
DKIM	Adds a digital signature to verify emails weren't tampered with
DMARC	Policy for handling emails that fail SPF/DKIM checks

Example SPF record:

```
"v=spf1 include:_spf.google.com ~all"
```

NS Record

Declares which name servers are authoritative for your domain. Auto-created by Route 53.

PTR Record

The opposite of an A record — maps an IP address back to a domain name. Used in reverse DNS lookups.

Normal: mywebsite.com → 54.12.34.56 (A Record)
Reverse: 54.12.34.56 → mywebsite.com (PTR Record)

Common uses: Email spam filtering, server logging, network debugging.

SRV Record

Specifies both the hostname AND port number for a specific service. Helps clients automatically discover services.

Format:	Priority	Weight	Port	Target
	10	20	5060	sip.mywebsite.com

Used in: VoIP, gaming servers, SIP, chat protocols (XMPP).

★ Alias Records

A Route 53 exclusive feature. Works like a CNAME but with superpowers — specifically designed for AWS services.

mywebsite.com → my-load-balancer-123.us-east-1.elb.amazonaws.com

Supported AWS targets:

- Application Load Balancer (ALB)
- CloudFront distributions
- API Gateway
- S3 static website buckets
- Global Accelerator

🔴 CNAME vs Alias — Side by Side

Feature	CNAME Record	Alias Record
Works on root domain (example.com)	❌ No	✅ Yes
Extra DNS query needed	Yes	No (resolved internally)
AWS IP changes handled automatically	❌ No	✅ Yes
Cost for AWS resource queries	Charged	Free
Health check support	Limited	✅ Native
Works outside AWS	✅ Yes	❌ AWS targets only

Rule of thumb: Always prefer Alias records when pointing to AWS resources.

🚦 Traffic Routing Strategies

Route 53 routing policies control **which answer DNS returns** when a user queries your domain. DNS does not route actual network packets — it only decides what IP or endpoint to tell the

user about.

All Policies at a Glance

Policy	Decision Factor	Health Checks	Best For
Simple	None (just returns value)	✗	Single-server basics
Weighted	Percentage split	✓	Gradual rollouts, A/B tests
Latency	Fastest response time	✓	Global performance
Failover	Primary or backup	✓ Required	Disaster recovery
Geolocation	User's country/region	✓	Localization
Geoproximity	Location + bias value	✓	Fine-tuned traffic shaping
Multi-Value	Multiple healthy IPs	✓	Basic distribution
IP-Based	Client IP / CIDR block	✓	ISP or enterprise routing

1 Simple Routing

The most straightforward policy — one domain, one destination.

```
user queries example.com
      ↓
Route 53 returns → 54.12.34.56
```

If you add multiple IPs, Route 53 returns all of them and the browser picks one randomly.

Limitation: **No** smart failover. **If** server goes down,
Route 53 still returns that IP ✗

Best for: Personal projects, single-server applications, testing environments.

2 Weighted Routing

Distributes traffic across multiple servers using assigned weight values.

Traffic Split Example:

Server A (New Version) → Weight 10 → Gets 10% of users

Server B (Old Version) → Weight 90 → Gets 90% of users

The math:

Server weight ÷ Total weight = % of traffic

Weight 70 **out** of 100 total = 70% traffic

Weight 30 **out** of 100 total = 30% traffic

Weight Value	Behavior
Any positive number	Proportional traffic share
0	No traffic sent to this resource
All records = 0	Traffic distributed equally

Best for: Blue-green deployments, canary releases, gradual feature rollouts.

3 Latency Routing ⚡

Sends each user to whichever AWS region gives them the fastest response — not necessarily the closest geographically.

A user **in** Singapore queries example.com:

Route 53 checks:

Singapore → ap-southeast-1 latency: 8ms ← Winner ✅

US East → us-east-1 latency: 190ms

Ireland → eu-west-1 latency: 220ms

Result: User routed **to** Singapore region

Geography ≠ Latency. A user in Germany might get lower latency from a US server than a European one depending on network conditions.

Best for: Global applications where user experience and response speed matter.

4 Failover Routing 🛡️

Maintains a primary server and a standby backup. Automatically switches when primary becomes unhealthy.

Normal State:

User → Primary Server (Healthy ✅) → Serves traffic

Failure State:

User → Primary Server (Down ❌)

→ Route 53 detects via health check

→ Secondary Server (Standby) → Takes over ✅

Record Role	Configuration	When Used
Primary	Failover = Primary	Always (when healthy)
Secondary	Failover = Secondary	Only when primary fails

Requires: Active health checks on the primary record.

Best for: Disaster recovery setups, high-availability critical applications.

5 Geolocation Routing 📍

Routes traffic based on where the user physically is located.

User in India → Routed to Mumbai server

User in Germany → Routed to Frankfurt server

User in Brazil → Routed to Default server (no Brazil-specific record)

Supported location granularity:

- Entire continent (e.g., Europe, Asia)
- Specific country (e.g., India, USA)
- US States specifically

Always create a **Default** record to catch users from unspecified locations.

Best for: Serving localized content, regional pricing, media licensing restrictions, GDPR compliance.

6 Geoproximity Routing 🗺️

Similar to Geolocation but adds a **bias** — letting you artificially expand or shrink which users get routed to which region.

Without Bias:

With Positive Bias (+50) on US-East:

EU → Europe

EU users near the boundary → US-East

US → US-East

US → US-East (even larger share)

Bias Setting	Traffic Impact
+1 to +99	Pull more users toward this region
-1 to -99	Push users away from this region
0	Default (no adjustment)

Requires **Route 53 Traffic Flow** to configure. More complex but very powerful.

Best for: Situations where you want business logic to override pure geography.

7 Multi-Value Answer Routing 🔄

Returns multiple healthy IP addresses in response to a single query. The client (browser/application) picks one.

Query **for** example.com returns:

54.12.34.56 (Healthy ✅)

54.12.34.99 (Healthy ✅)

54.12.34.11 (Unhealthy ❌ – excluded automatically)

- Returns up to **8 healthy records** per query
- Each record can have its own health check
- Unhealthy records are automatically filtered out

This is **not** the same as a Load Balancer. It provides basic distribution but lacks advanced features like session persistence.

Best for: Simple distribution across multiple EC2 instances or servers.

8 IP-Based Routing 🌐

Routes traffic based on the client's source IP address or CIDR block.

Users **from** 203.0.113.0/24 → Server A (e.g., ISP-specific optimized server)

Users **from** 198.51.100.0/24 → Server B

Everyone **else** → Default server

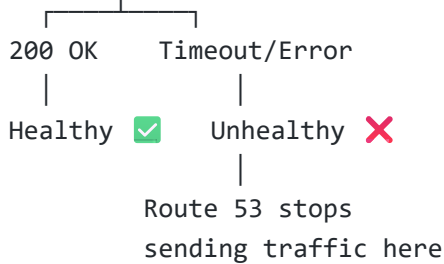
Best for: Routing specific ISPs, enterprise networks, or branch offices to dedicated endpoints.

♥ Health Monitoring in Route 53

Route 53 Health Checks continuously probe your endpoints and mark them as healthy or unhealthy — automatically influencing routing decisions.

Route 53 Health Checkers (15 globally distributed)

↓
Send HTTP/HTTPS/TCP probe to your endpoint



Three Types of Health Checks

◆ Type 1 — Endpoint Monitoring

Directly tests a URL, IP, or domain using HTTP, HTTPS, or TCP.

Configuration example:

```

Protocol   : HTTPS
Endpoint   : api.mywebsite.com
Port       : 443
Path       : /health
Interval   : 30 seconds
Threshold  : 3 failures = unhealthy
  
```

◆ Type 2 — Calculated Health Check

Combines results from multiple child health checks using logic operators.

```

Parent Check = Healthy IF:
  (Check-A AND Check-B) OR (Check-C)
  
```

Useful for: Treating a resource as healthy only when multiple conditions are met.

◆ Type 3 — CloudWatch Alarm Health Check

Uses an existing CloudWatch Alarm as the health signal instead of directly probing.

CloudWatch Alarm (CPU > 90% for 5 min)



ALARM state



Route 53 marks endpoint as Unhealthy ❌

Essential for **private VPC resources** — Route 53 health checkers are public and cannot reach private IPs directly.

✅ What Counts as Healthy?

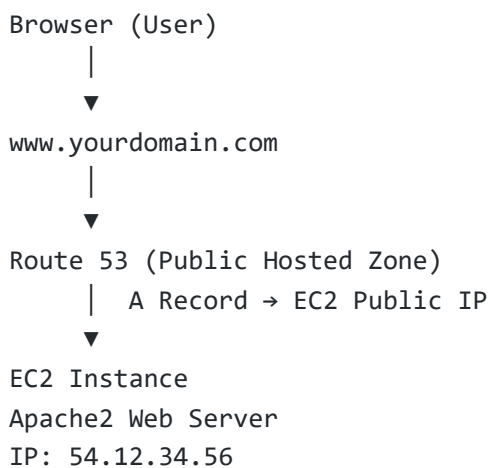
Response	Status
HTTP 2xx or 3xx	✅ Healthy
HTTP 4xx, 5xx	❌ Unhealthy
Connection timeout	❌ Unhealthy
Optional: Response body match	Must contain expected string

🔧 Hands-On Labs

🧪 Lab 1 — Host a Website with Simple Routing

Goal: Take an EC2 instance, install a web server, and make it accessible via a custom domain through Route 53.

Architecture Diagram



🔧 Step 1 — Get a Domain Name

Option A — Buy from Namecheap / GoDaddy:

- Purchase your preferred domain
- You'll update nameservers later to point to Route 53

Option B — Register directly in Route 53:

- AWS Console → Route 53 → Register Domain
- Hosted zone gets created automatically

🔧 Step 2 — Launch EC2 Instance

Configuration	Value
AMI	Ubuntu 22.04 LTS
Instance Type	t2.micro (free tier)
Auto-assign Public IP	Enable
Security Group — Inbound	SSH (port 22), HTTP (port 80)
Key Pair	Create or use existing

1. Go to EC2 → Launch Instance
2. Configure settings above
3. Launch and note down the **Public IPv4 address**

🔧 Step 3 — Set Up Web Server

SSH into the instance:

```
ssh -i your-key.pem ubuntu@YOUR_EC2_PUBLIC_IP
```

Install and start Apache2:

```
sudo apt update -y
sudo apt install apache2 -y
sudo systemctl start apache2
sudo systemctl enable apache2
```

Create a test webpage:

```
sudo bash -c 'cat > /var/www/html/index.html << EOF
<!DOCTYPE html>
<html>
  <head><title>My Route 53 Site</title></head>
  <body>
    <h1>🚀 Website Live via AWS Route 53!</h1>
    <p>DNS is working correctly.</p>
  </body>
</html>
EOF'
```

Quick verify — paste EC2 public IP in browser:

http://YOUR_EC2_PUBLIC_IP

You should see your webpage.

🔧 Step 4 — Create Route 53 Hosted Zone

1. AWS Console → Route 53 → Hosted Zones
2. Click **Create Hosted Zone**

Field	Value
Domain Name	yourdomain.com
Type	Public Hosted Zone

3. After creation, copy the **4 NS record values** shown

Example NS values:

```
ns-388.awsdns-48.com
ns-1412.awsdns-48.org
ns-763.awsdns-31.net
ns-1659.awsdns-15.co.uk
```

🔧 Step 5 — Point Domain to Route 53

For Namecheap:

- Dashboard → Domain List → Manage
- Nameservers section → Select **Custom DNS**
- Paste all 4 Route 53 NS values

For GoDaddy:

- My Products → DNS Settings
- Nameservers → Change → Select **I'll use my own nameservers**
- Paste all 4 Route 53 NS values

⌚ DNS propagation can take anywhere from a few minutes to 48 hours globally.

🔧 Step 6 — Create the A Record

1. Route 53 → Your Hosted Zone → **Create Record**

Field	Value
Record Name	www (or leave blank for root)
Record Type	A
Routing Policy	Simple
Value	Your EC2 Public IP
TTL	300

2. Save the record

🔧 Step 7 — Test & Validate**Command line checks:**

```
# Check what IP the domain resolves to
nslookup yourdomain.com
```

```
# Detailed DNS lookup
dig yourdomain.com
```

```
# Check from a specific DNS server
dig @8.8.8.8 yourdomain.com
```

Browser test:

<http://yourdomain.com>

✅ You should see your custom webpage served from EC2!

Lab 2 — Weighted Routing (Traffic Splitting)

Goal: Split traffic between two servers — e.g., 80% to stable version, 20% to new version.

Setup — Two EC2 instances with different pages:

On Server 1:

```
echo "<h1>Server 1 – Stable Version</h1>" | sudo tee /var/www/html/index.html
```

On Server 2:

```
echo "<h1>Server 2 – New Version (Beta)</h1>" | sudo tee /var/www/html/index.html
```

Route 53 Records:

Record	IP	Weight	Set ID
example.com	Server1-IP	80	server-stable
example.com	Server2-IP	20	server-beta

Verify traffic split:

```
for i in {1..10}; do
  dig +short yourdomain.com
  sleep 1
done
```

You should see Server 1's IP appear ~8 times and Server 2's ~2 times.

Lab 3 — Latency Routing

Goal: Route users to the AWS region that gives them the lowest latency.

Setup — EC2 instances in multiple regions:

Region	Location	Instance IP
ap-south-1	Mumbai	IP-Mumbai
us-east-1	N. Virginia	IP-Virginia
eu-west-1	Ireland	IP-Ireland

Route 53 Records:

For each region, create a record with:

Field	Value
Record Type	A
Routing Policy	Latency
Region	Select the EC2's AWS region
Value	That region's EC2 IP
Health Check	Attach one

Route 53 automatically routes each user to their lowest-latency region.

🔧 Lab 4 — Failover Routing

Goal: Keep a backup server ready to take over if the primary goes down.

Create a Health Check First

Route 53 → Health Checks → Create Health Check

Field	Value
Name	primary-server-check
Monitor	Endpoint
Protocol	HTTP
IP / Domain	Primary server IP
Port	80
Path	/health
Interval	30 seconds
Failure Threshold	3

Create Health Check Endpoint on Primary Server:

```
sudo mkdir -p /var/www/html/health
echo "OK" | sudo tee /var/www/html/health/index.html
```

Create Failover DNS Records

Primary Record:

Field	Value
Record Type	A
Routing Policy	Failover
Failover Type	Primary
Value	Primary server IP
Health Check	primary-server-check

Secondary Record:

Field	Value
Record Type	A
Routing Policy	Failover
Failover Type	Secondary
Value	Backup server IP

Test Failover

Simulate primary server failure:

```
# On primary server – stop Apache  
sudo systemctl stop apache2
```

Wait ~90 seconds for health check failure threshold to trigger, then:

```
dig +short yourdomain.com
```

Should now return the **secondary server's IP** ✅

Restore primary:

```
sudo systemctl start apache2
```

Goal: Serve different content to users based on their country.

Setup servers in each target region and create records:

Record	Geolocation	IP	Description
example.com	India	India-Server-IP	Indian users
example.com	United States	US-Server-IP	US users
example.com	Default	Global-Server-IP	Everyone else

Without a Default record, users from unmapped locations get a "no response" error.

🧪 Lab 6 — Multi-Value Routing

Goal: Return multiple healthy IPs so clients can pick any available server.

Create separate A records for each server:

Field	Record 1	Record 2	Record 3
Name	example.com	example.com	example.com
Type	A	A	A
Policy	Multivalue Answer	Multivalue Answer	Multivalue Answer
Value	IP-1	IP-2	IP-3
Health Check	Check-1	Check-2	Check-3

Test — multiple IPs returned:

```
dig yourdomain.com
# Should show up to 8 IPs in the answer section
```

If one server goes down, its IP is automatically excluded from responses.

📄 Summary & Cheat Sheet

📁 DNS Record Quick Reference

Record	Maps	Example
A	Domain → IPv4	site.com → 1.2.3.4
AAAA	Domain → IPv6	site.com → 2001:db8::1
CNAME	Domain → Domain	www.site.com → site.com
MX	Domain → Mail server	site.com → mail.google.com
TXT	Domain → Text	SPF, DKIM, verification
NS	Domain → Name servers	Auto-managed by Route 53
PTR	IP → Domain	Reverse lookup
SRV	Service → Host:Port	VoIP, gaming
Alias	Domain → AWS resource	site.com → ALB endpoint

🚦 Routing Policy Quick Picker

Need basic single server?	→ Simple Routing
Rolling out a new feature?	→ Weighted Routing
Global app, speed matters?	→ Latency Routing
Need a backup if server fails?	→ Failover Routing
Different content per country?	→ Geolocation Routing
Custom traffic shaping?	→ Geoproximity Routing
Multiple servers, basic spread?	→ Multi-Value Routing
Route by network/ISP?	→ IP-Based Routing

💡 Key Concepts to Remember

- ✅ Alias records are FREE **for** AWS targets
- ✅ CNAME cannot be used at root domain – use Alias instead
- ✅ Health checks are REQUIRED **for** Failover routing
- ✅ TTL affects how quickly DNS changes take effect
- ✅ Private hosted zones only work inside VPCs
- ✅ Route 53 health checkers are public – cannot reach private IPs
- ✅ Latency routing ≠ Geolocation routing
- ✅ Multi-Value is **NOT** a replacement **for** a Load Balancer
- ✅ DNS resolves names – it does **not** route actual traffic packets
- ✅ Always add a Default record **in** Geolocation routing